

Part 2 进程描述



学习目标

- 能够解释进程引入的原因
- 能够从并发性、动态性、独立性、异步性描述进程的特征
- 能够理解并解释进程的组成
- 能够解释进程的基本状态并举例分析状态转换

Part 2 进程描述

2.1 进程引入

- **进程定义**

- 进程是**程序**的**一次**执行。
- 进程是一个**程序**及其**数据**在处理机上顺序执行时所发生的**活动**。
- 进程是**程序**在一个**数据集合**上运行的**过程**，它是系统进行**资源分配**和**调度**的一个**独立单位**。
- 行为的**规则**叫程序，程序在处理机上执行的**活动**称为进程

Part 2 进程描述

2.1 进程引入

- 进程和程序区别

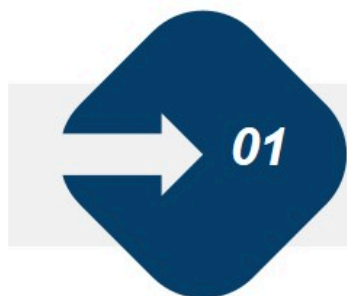


Part 2 进程描述

2.1 进程引入

● 进程特征

动态性 (最基本的特征)



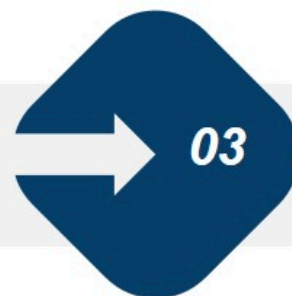
➤ 生命期

并发性



➤ 一段时间内
同时运行

独立性



➤ 进程实体是一个能独立运行的基本单位
➤ 是系统中独立获得资源和独立调度的基本单位

异步性



➤ 按各自独立的、不可预知的速度向前推进

Part 2 进程描述

2.1 进程引入

- **进程组成**

程序+数据+进程控制块

- ❖ **程序段**

- ❖ **数据段**

- ❖ **进程控制块** (Process Control Block) **PCB**: 用于描述进程情况及控制进程运行所需的全部信息。

2.2 进程状态1——3种基本状态

- **3种基本状态**

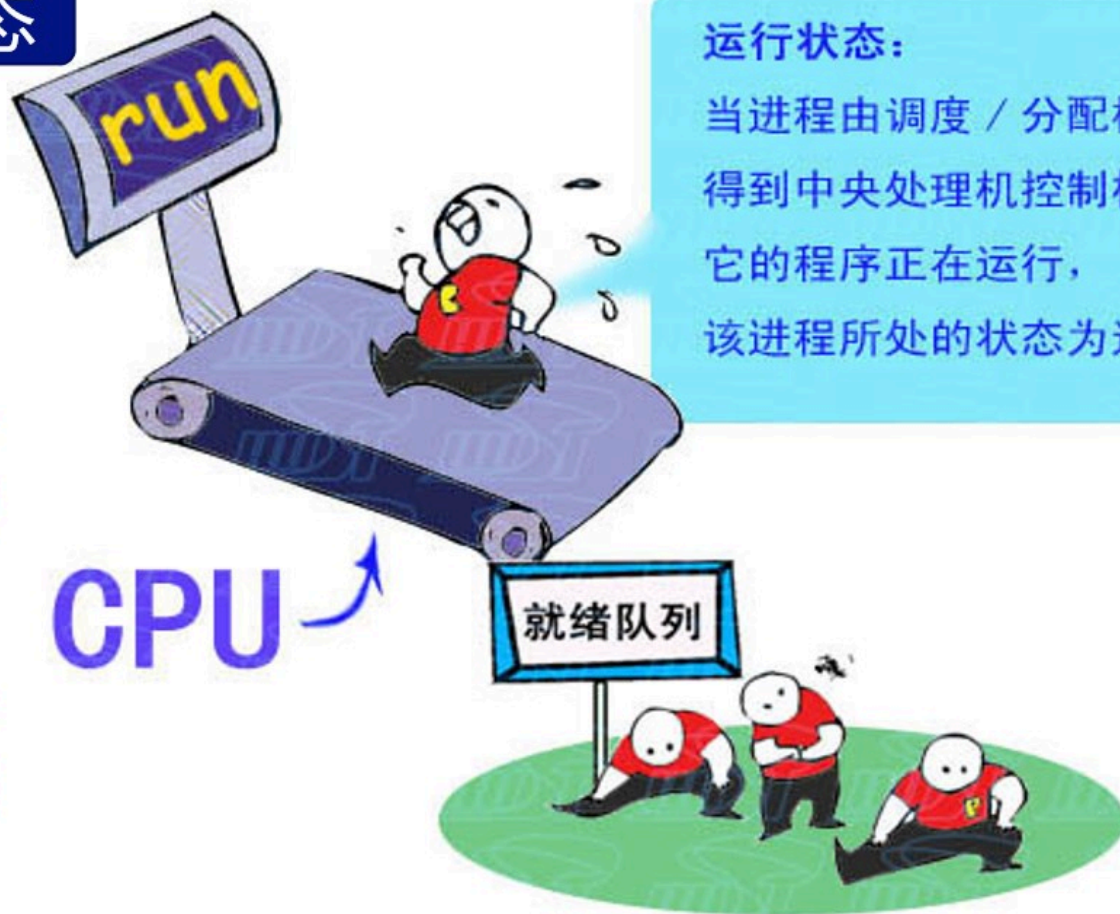
- 执行状态 (Running) (运行状态)
- 就绪状态 (Ready)
- 阻塞状态 (Block) (等待状态)

Part 2 进程描述

2.2 进程状态

执行状态:

- 单处理机:
一个进程处于执行状态
- 多处理机:
多个进程处于执行状态



运行状态:

当进程由调度 / 分配模块分配后，
得到中央处理机控制权，
它的程序正在运行，
该进程所处的状态为运行状态。

Part 2 进程描述

2.2 进程状态

就绪状态:

- 不会有CPU空闲用户进程在就绪的情况

就绪状态:

存在预处理机调度队列中的那些进程，他们已经准备就绪，一旦得到 CPU 控制权，就立即可以运行，这些进程所处的状态为就绪状态。

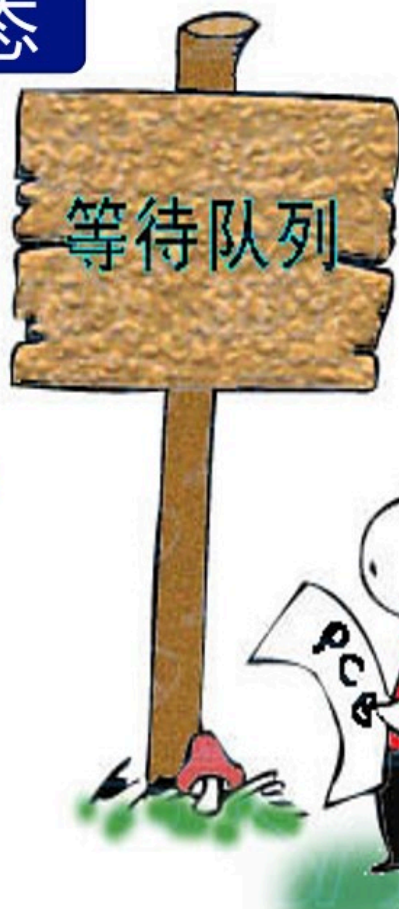


Part 2 进程描述

2.2 进程状态

阻塞（等待） 状态：

- 典型事件：
请求I/O、申请缓冲空间
- 根据阻塞原因，设置多个阻塞队列



等待状态：

若一进程正在等待着某一事件发生而暂时停止执行，这时，即使给它 CPU 控制权，它也无法执行，则称该进程处于等待状态，又可称阻塞状态。

Part 2 进程描述

2.2 进程状态

状态转换



Part 2 进程描述

2.2 进程状态

■ Q3: 下列两种状态转换有没有?

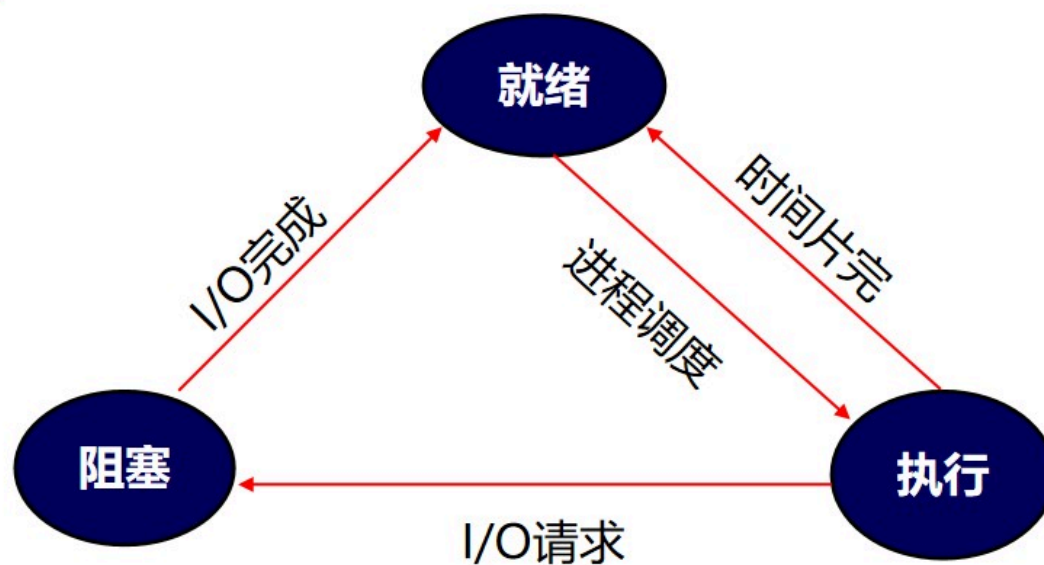
❖ 就绪 → 等待?

❖ 等待 → 执行?

Part 2 进程描述

2.2 进程状态

状态转换



Part 2 进程描述

2.2 进程状态

例题

一个只有一个处理机的系统中，OS的进程有运行、就绪、阻塞三个基本状态。假如某时刻该系统中有10个进程并发执行，在略去调度程序所占用时间情况下试问：

- 这时刻系统中处于运行态的进程数最多有几个？最少有几个？
- 这时刻系统中处于就绪态的进程数最多有几个？最少有几个？
- 这时刻系统中处于阻塞态的进程数最多有几个？最少有几个？

1 0
9 0
10 0

Part 2 进程描述

2.2 进程状态

例题

某处理器有4个核，目前系统中若同时存在10个进程并发执行，在略去调度程序所占用时间情况下试问：

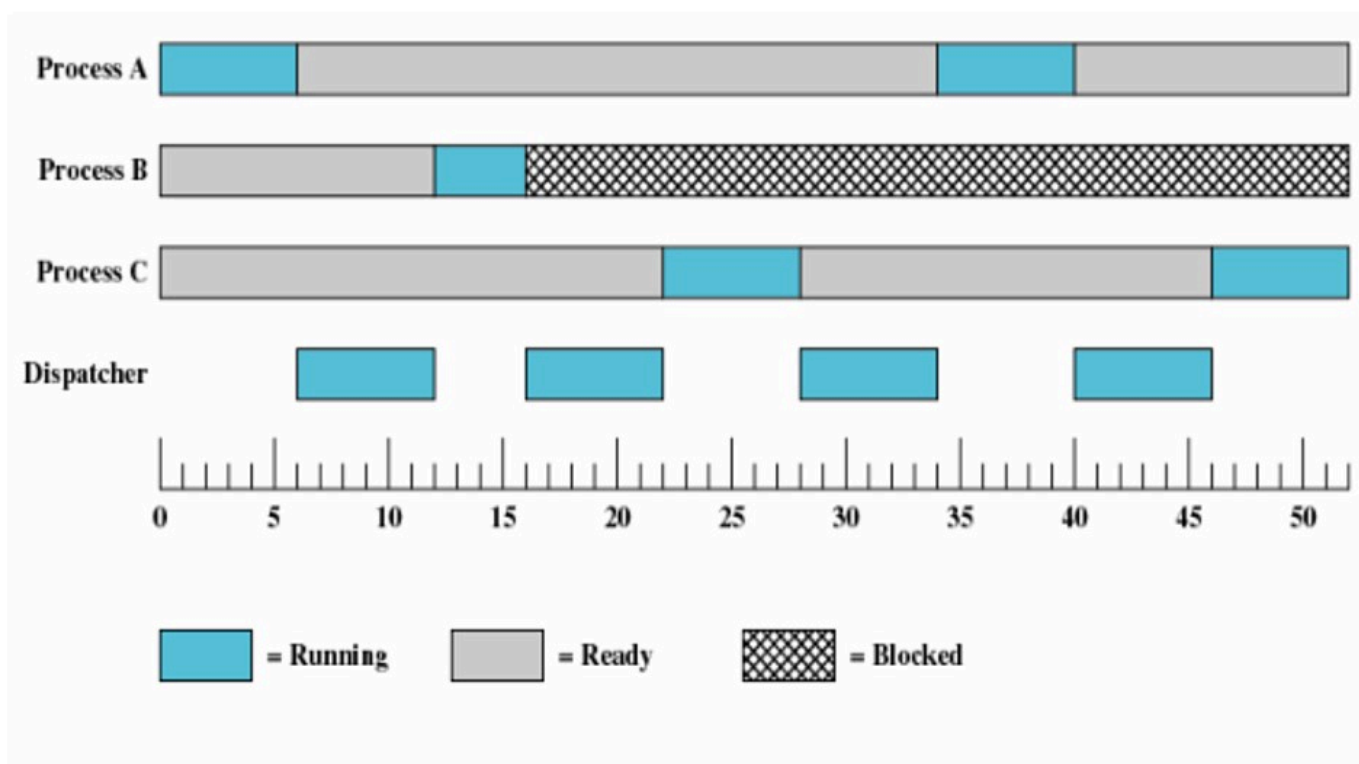
- 处于运行态的进程数最多有几个？最少有几个？
- 处于就绪态的进程数最多有几个？最少有几个？
- 处于阻塞态的进程数最多有几个？最少有几个？

4
6
10

0
0
0

Part 2 进程描述

2.2 进程状态



Part 2 进程描述

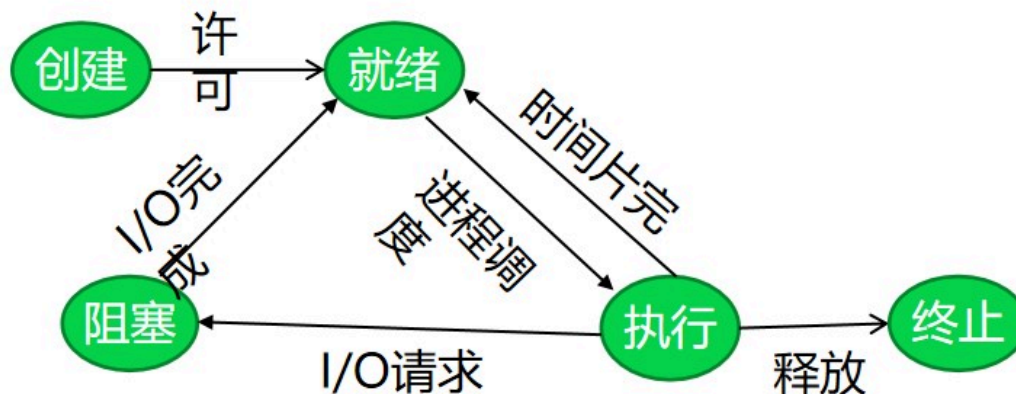
2.2 进程状态2——5种基本状态

01 创建状态

- 申请一个空白PCB；填写PCB；分配资源；设置就绪状态插入就绪队列

02 终止状态

- 等待OS善后；
- 收回PCB



Part 2 进程描述

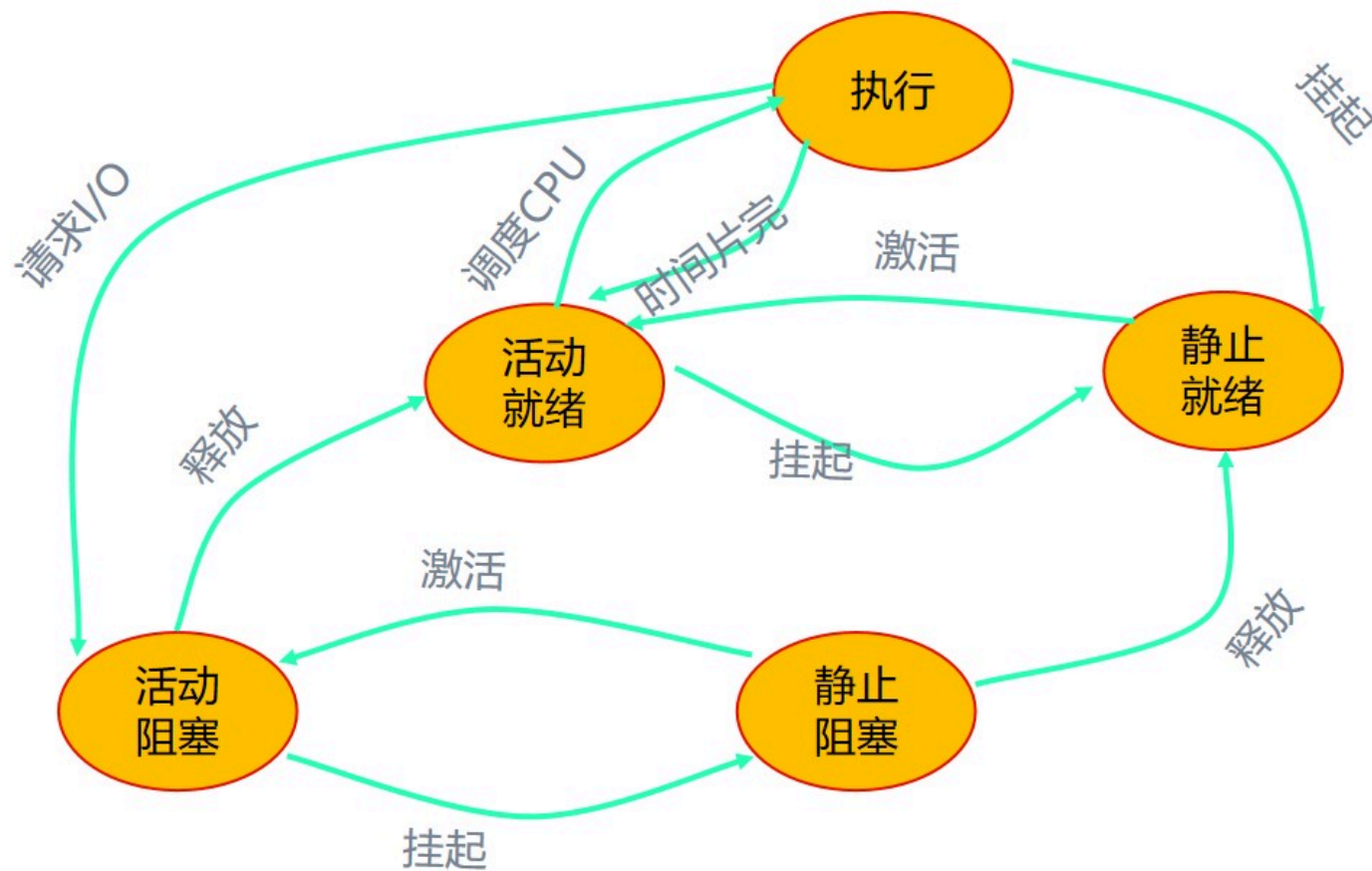
2.2 进程状态3——7种基本状态

挂起状态

当终端用户在自己的程序运行期间，有时希望暂时使自己的进程静止下来。如果进程处于执行状态，则暂停执行；如果进程处于就绪状态，则暂时不接受调度，以便研究其执行情况或对程序进行修改。

Part 2 进程描述

2.2 进程状态3——7种基本状态



Part 2 进程描述

2.3 进程控制块

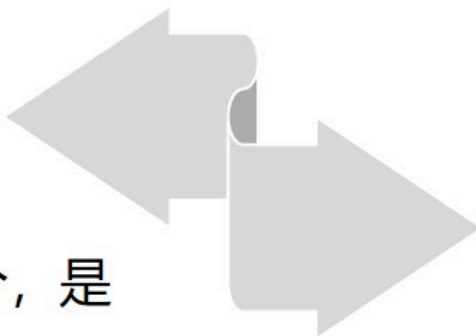
- 操作系统中用于**管理控制**的数据结构
 - ❖ 进程：PCB.....
 - ❖ 内存：分区表，页表.....
 - ❖ 设备：设备控制表，控制器控制表.....
 - ❖ 文件：文件控制块，目录，索引表.....

Part 2 进程描述

2.3 进程控制块



PCB是进程的一部分，是操作系统中最重要的**记录型数据结构**，是进程存在的唯一标志，常驻内存。



PCB的作用：

- 作为独立运行基本单位的标志；
- 能实现间断性运行方式；
- 提供进程管理所需要的信息；
- 提供进程调度所需要的信息；
- 实现与其他进程的同步与通信。

Part 2 进程描述

2.3 进程控制块

PCB的信息



- 进程标识符
- 处理机状态
- 进程调度信息
- 进程控制信息

Part 2 进程描述

2.3 进程控制块



PCB的信息——处理机状态信息

处理机状态信息主要是由处理机各种寄存器中的内容所组成。

➤通用寄存器

➤指令寄存器。存放要访问的下一条指令。

➤程序状态字PSW。条件码、执行方式、中断屏蔽标志等

➤用户栈指针。每个用户进程有一个或若干个与之相关的系统栈，用于存放过程和系统调用参数及调用地址。栈指针指向该栈的栈顶。

Part 2 进程描述

2.3 进程控制块



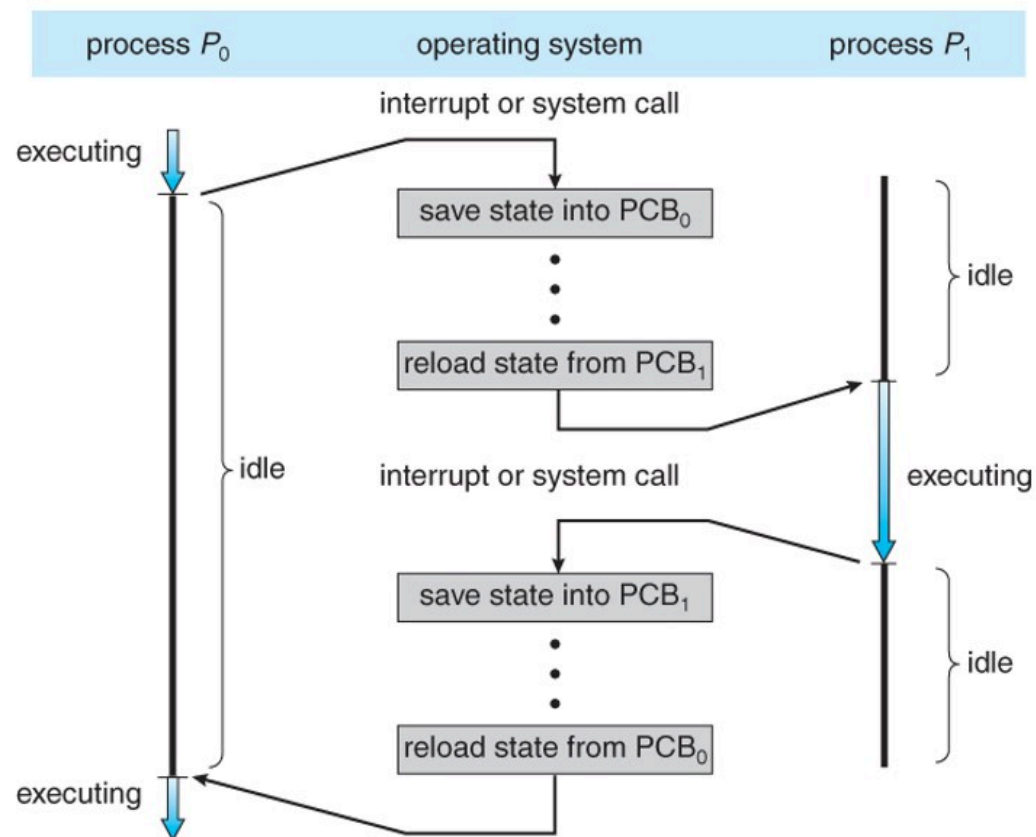
PCB的信息——进程控制信息

- 程序和数据的地址
- 进程同步和通信机制
- 资源清单
- 链接指针

Part 2 进程描述

2.3 进程控制块

进程切换时:



Part 2 进程描述

2.3 进程控制块

PCB表

系统把所有PCB组织在一起，并把它们放在内存的固定区域，就构成了PCB表。

PCB表的大小决定了系统中最多可同时存在的进程个数，称为系统的并发度。

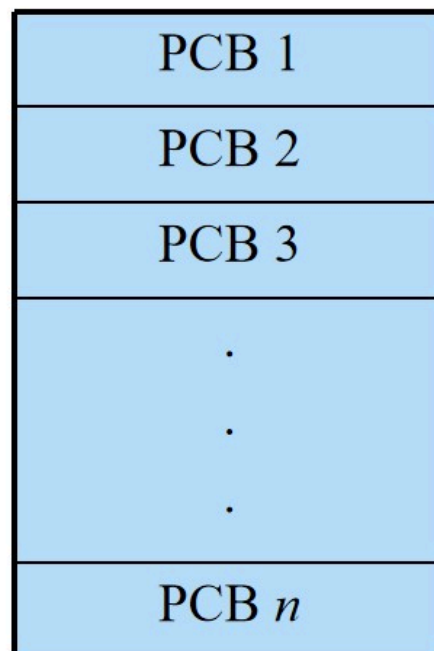
PCB表组织方式

- 线性方式
- 链接方式
- 索引方式

Part 2 进程描述

2.3 进程控制块

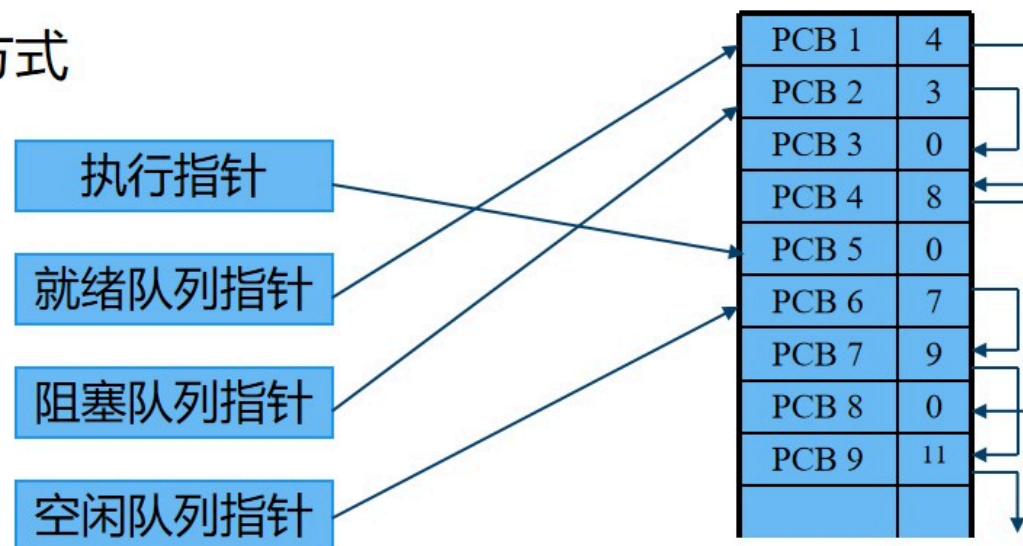
线性组织方式



Part 2 进程描述

2.3 进程控制块

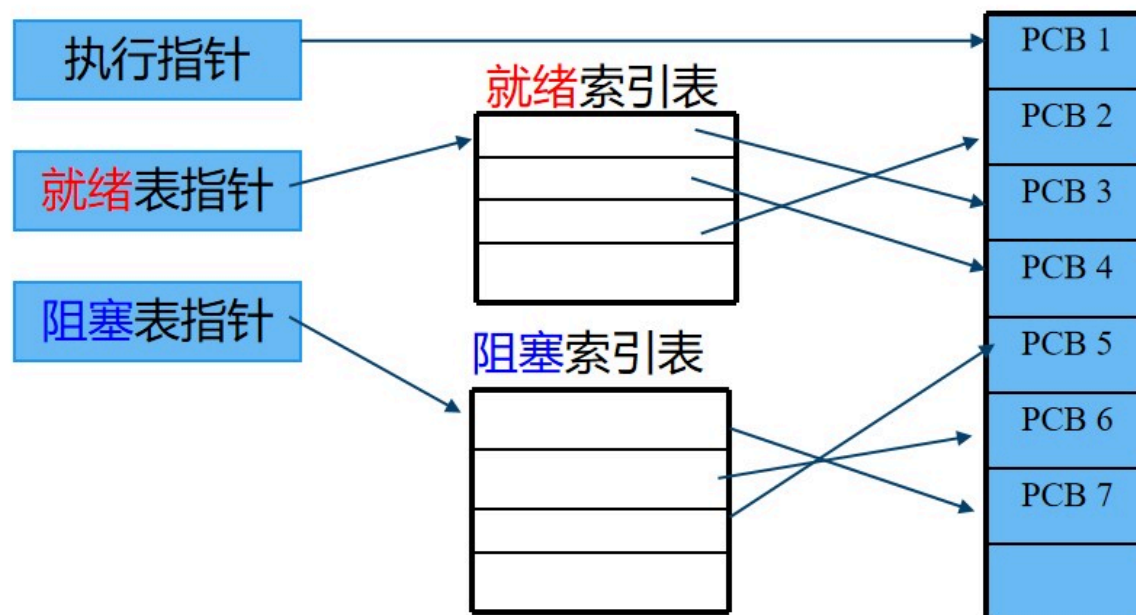
链接组织方式



Part 2 进程描述

2.3 进程控制块

索引组织方式



Part 2 进程描述

2.4 Unix进程实现

- Unix进程定义

- ❖ UNIX的进程由三部分组成:

- ❖ proc结构(常驻内存的PCB)

- ❖ 数据段(执行时用到的数据)

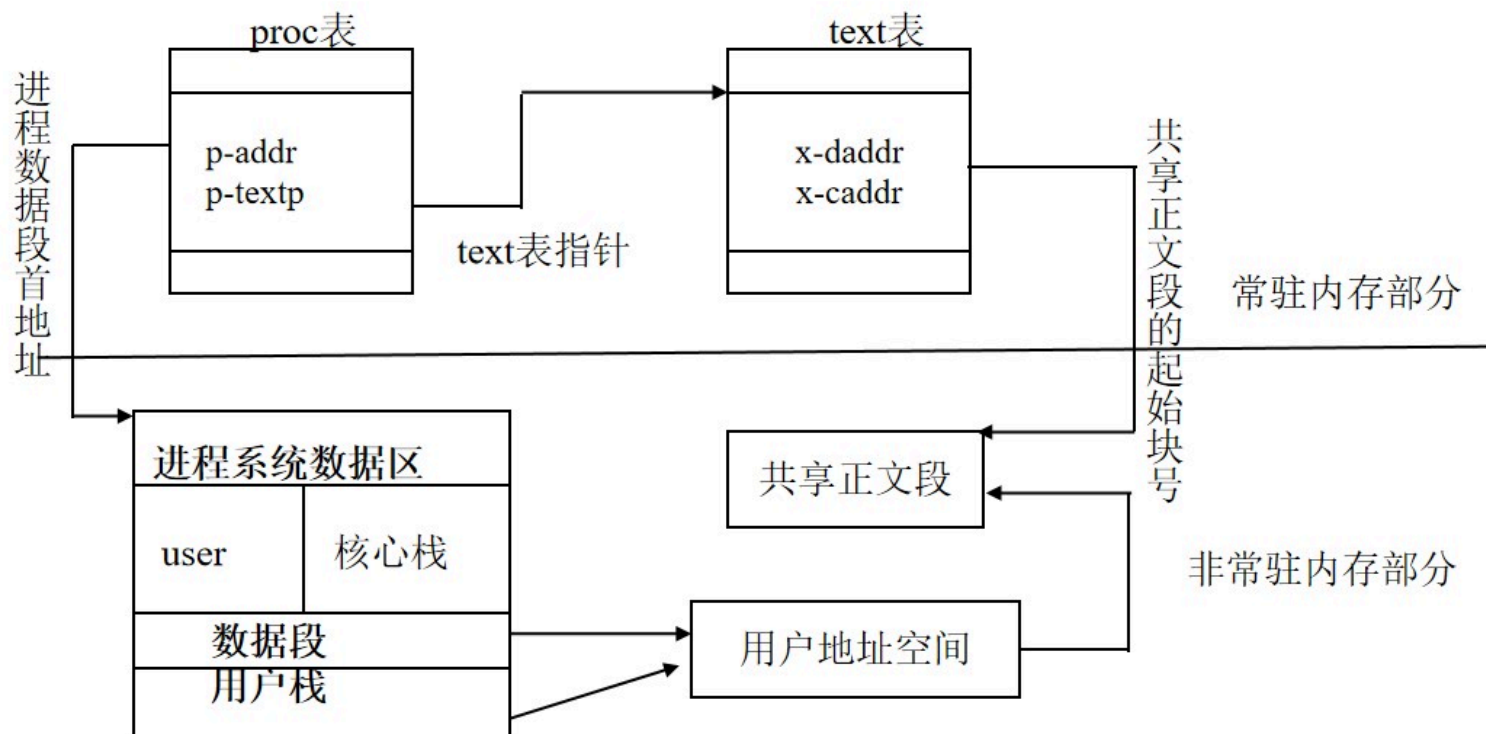
- ❖ 正文段(程序代码)

合称进程映像, UNIX中把进程定义为映像的执行。

Part 2 进程描述

2.4 Unix进程实现

● Unix进程定义



Part 2 进程描述

2.4 Unix进程实现

● Unix进程定义

(1) 进程基本控制块结构

```
struct proc{
    char p_stat;           /*进程状态*/
    char p_flag;           /*进程标志*/
    char p_pri;            /*进程优先级*/
    char p_sig;            /*软中断号*/
    char p_uid;            /*用户号*/
    char p_time;           /*驻留时间*/
    char p_cpu;            /*进程占据CPU的时间量*/
    char p_nice;           /*用于计算优先级*/
    int p_ttyp;            /*控制终端tty结构的地址*/
    int p_pid;             /*进程号*/
    int p_ppid;            /*父进程号*/
    int p_addr;            /*数据段地址*/
    int p_size;            /*数据段大小*/
    int p_wchan;           /*等待的原因*/
    int *p_textp;          /*对应正文段的text项地址*/
}proc[NPROC];
```

Part 2 进程描述

2.4 Unix进程实现

● Unix进程定义

(2) 进程扩展控制块结构

```
struct user{
    int u_rsav[2];           /*保留现场保护区指针*/
    char u_segflg;          /*用户/核心空间标志*/
    char u_error;           /*返回出错代码*/
    char u_uid;             /*有效用户号*/
    char u_gid;             /*有效组号*/
    int u_procp;            /*proc结构地址*/
    char u_base;            /*内存地址*/
    char *u_count;          /*传送字节数*/
    char *u_offset[2];      /*文件读写位移*/
    int *u_cdir;            /*当前目录I节点地址*/
    char *u_dirp;           /*I节点当前指针*/
    Struct
    {
        int u_ino;
        char u_name [DIRSIZ];
    }u_dent;                /*当前目录项*/
}
```

Part 2 进程描述

2.4 Unix进程实现

- Unix进程定义

(2) 进程扩展控制块结构 (续)

```
int u_ofile  [NOFILE];    /*用户打开文件表*/
int u_arg[5];             /*存系统调用的自变量*/
int u_tsize;              /*正文段大小*/
int u_dsize;              /*用户资料区大小*/
int u_ssize;              /*用户栈大小*/
int u_ftime;              /*用户态执行时间*/
int u_stime;              /*核心态执行时间*/
int u_cutime;             /*子进程用户态执行时间*/
int u_cstime;             /*子进程核心态执行时间*/
int u_ar0;                /*当前中断保护区内r0地址*/
};
```

Part 2 进程描述

2.4 Unix进程实现

- Unix进程定义

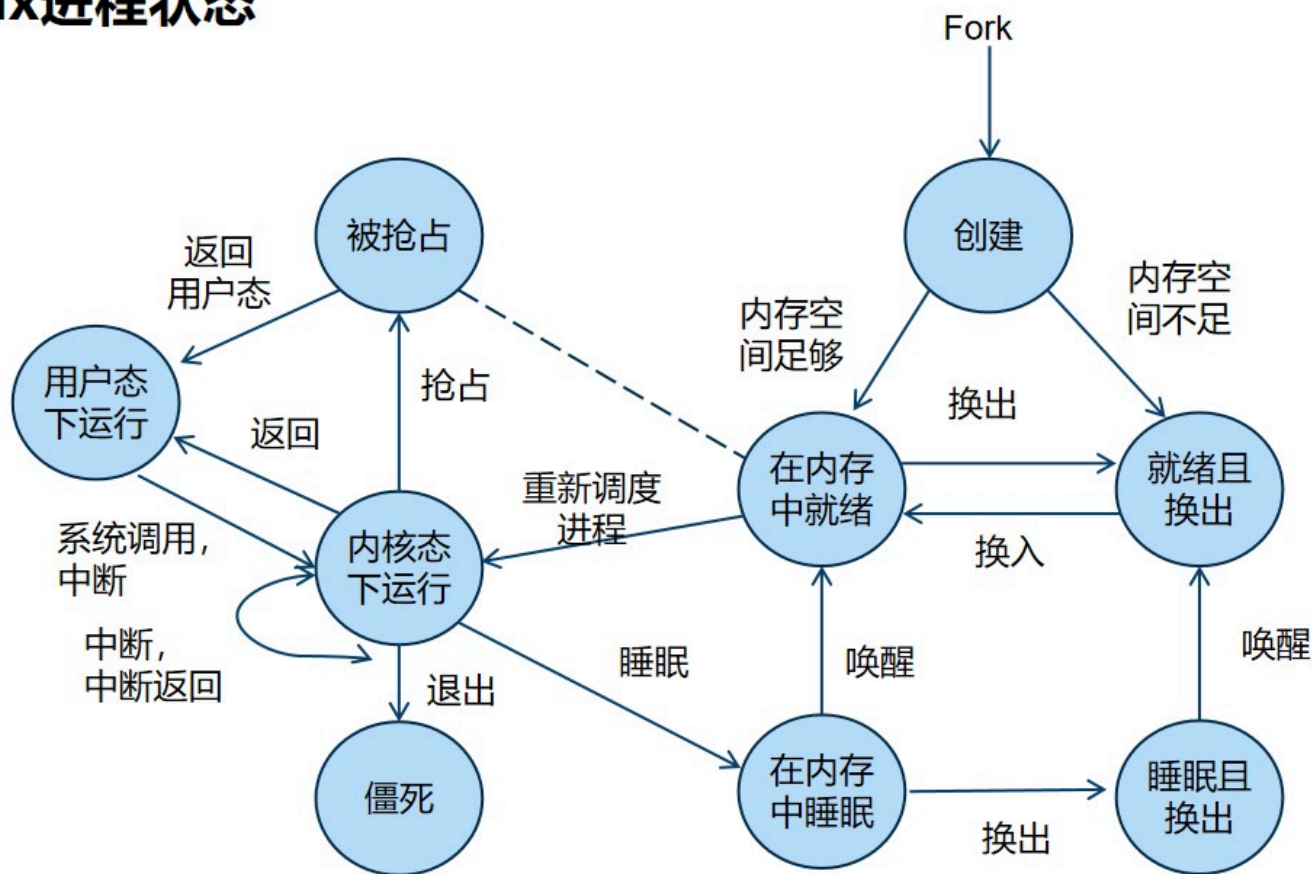
(3) 正文段结构

```
struct text{  
    int x_daddr;           /*磁盘地址*/  
    int x_caddr;           /*内存地址*/  
    int x_size;            /*内存块数，每块64字节*/  
    int x_iptr;            /*文件内存I节点地址*/  
    char x_count;          /*共享进程数*/  
    char x_ccount;         /*内存副本的共享进程数*/  
}text[NTEXT];
```

Part 2 进程描述

2.4 Unix进程实现

● Unix进程状态



Part 2 进程描述



目标评价

1. 解释为什么并发执行的程序需要进程描述
2. 分析进程的并发、独立、动态、异步特征
3. 分析进程和程序的区别
4. 进程有哪几种状态？如何转换？
5. 进程的PCB有何作用？

关于进程和程序的区别描述不正确的是 ()。

- ☐ A 进程可以随着时间而变化 ✓
- ☒ B 程序也可以并发执行 ✗
- ☐ C 程序是进程存在的前提 ✓
- ☐ D 程序是进程执行的规则，进程是程序的执行过程 ✓

在一个单CPU系统中，若有6个用户进程。假设当前系统为用户态，则处于就绪状态的用户进程最多有 [填空1] 个，处于阻塞的进程最多有 [填空2] 个。

6

5

为了描述进程的动态变化过程，采用了一个与进程相联系的（ ），根据它而感知进程的存在及对进程进行控制。

- A 进程状态字
- B 进程优先数
- C 进程控制块
- D 进程起始地址

在进程管理中，当（ ），进程从阻塞状态变为就绪状态。

- ☐ A 进程被调度程序选中
- ☐ B 进程等待某一事件发生
- ☒ C 等待的事件出现
- ☐ D 时间片到

在分时系统中, 一个进程用完给它的时间片后, 其状态变为 ()

☒ A 就绪

☐ B 等待

☐ C 运行

☐ D 由用户设定

一个进程被唤醒，意味着该进程（）

- ☒ A 重新占有CPU
- ☐ B 优先级变为最大
- ☐ C 移至等待队列之首
- ☒ D 变为就绪状态

Part 3 进程控制



学习目标

- 能够理解进程创建、撤销、阻塞、唤醒的实现过程
- 能够理解Unix/Linux中fork、exec、wait、exit等进程控制函数的实现
- 能够应用Unix/Linux的进程控制函数分析和设计程序

Part 3 进程控制

3.1 原语

- 原语 (primitive)

有若干条指令组成，用于完成一定功能的一个过程。

- 进程管理最基本的功能：由OS内核中的原语实现

Part 3 进程控制

3.2 进程控制

- 进程创建
- 进程终止
- 进程阻塞和唤醒
- 进程挂起和激活

Part 3 进程控制

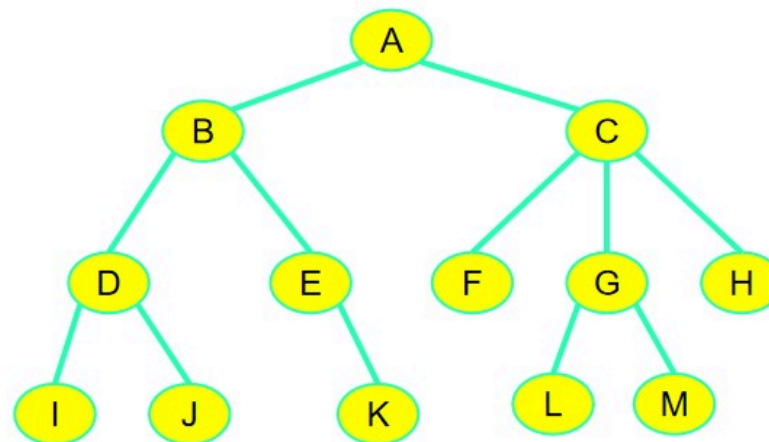
3.2 进程控制

● 进程创建

进程具有层次结构

引起进程创建的事件

- 用户登录、作业调度、提供服务、应用请求



Part 3 进程控制

3.2 进程控制

● 进程创建

进程创建过程

- ① 申请空白PCB;
- ② 分配所需资源;
- ③ 初始化PCB;
- ④ 插入就绪队列。



Part 3 进程控制

3.2 进程控制

- 进程终止

引起进程终止的事件

正常结束

异常结束：越界错误、保护错、特权指令错、非法指令错、运行超时、等待超时、算术运算错、I/O故障

外界干预

Part 3 进程控制

3.2 进程控制

- 进程终止

进程终止过程

找进程的PCB，读取进程状态

终止进程，并设置调度标志

终止所有子进程

回收进程所拥有的全部资源

回收PCB，生命周期结束



Part 3 进程控制

3.2 进程控制

● 进程阻塞与唤醒

引起进程阻塞的事件

- 请求系统服务
- 启动某操作
- 新数据尚未到达
- 无新工作可做

进程阻塞过程

- 停止执行
- 状态由执行改为阻塞
- 将PCB插入阻塞队列。

Part 3 进程控制

3.2 进程控制

● 进程阻塞与唤醒

进程唤醒过程

- 从阻塞队列中移出
- 状态由阻塞改为就绪
- 将PCB插入就绪队列。

Part 3 进程控制

3.2 进程控制

- 进程阻塞与唤醒

成对出现



Part 3 进程控制

3.2 进程控制

● 进程挂起与激活

进程的挂起

- Suspend()原语

进程的激活过程

- Active()原语

Part 3 进程控制

3.3 Unix进程控制实现

● 创建fork

引例2-3 Fork举例——C

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{ int pid;
```

```
    printf("the parent is going to fork\n");
```

```
    pid=fork();
```

运行结果? ?

```
if(pid!=0)
```

```
    printf("I am the father of %d\n",pid);
```

```
else
```

```
    printf("I am the child\n");
```

```
return 0;
```

```
}
```